Práctica 1.1 Arquitecturas y lenguajes de programación en clientes web

Resultados de Aprendizaje

- **RA1**: Selecciona las arquitecturas y tecnologías de programación sobre clientes Web, identificando y analizando las capacidades y características de cada una

Contenidos

1. Desarrollo web
    1. Conceptos básicos
    2. Áreas
    3. Front-end y back-end
2. Lenguajes de programación en clientes web
    1. Características
    2. Compatibilidades
    3. Seguridad
    4. Frameworks, APIs y librerías de desarrollo front-end
    5. Testing en front-end
3. Integración de código Javascript con HTML
    1. Estructura de carpetas
    2. HolaMundo.js
    3. Ofuscación de código
4. Herramientas y utilidades de programación
    1. Herramientas del navegador web
    2. Editor de texto e IDEs
        1. Visual Studio Code
    3. Node.js
    4. Gestión paquetes con NPM
    5. Detección de errores con ESlint
    6. Gestión de versiones con Github

1.- Descripción de la tarea

ACTIVIDAD 1. Empezando con el desarrollo web frontend:

Rellena la siguiente tabla describiendo los siguientes términos relacionados con el desarrollo Web. Para cada uno de ellos debes indicar su propósito, si pertenecen al lado servidor o cliente y al menos dos características diferenciadoras.

Término	Descripción y propósito	Cliente/Servidor	Característica 1	Característica 2
HTML5				
Javascript				
ECMAScript				
TypeScript				
PHP				
CSS				
SASS				
jQuery				
Node.js				
React.js				
AJAX				
Fetch API				
WebSocket				
Web Worker				

Crea una presentación multimedia (canva, presentaciones de google) para explicar **uno de los siguientes conceptos**: Fetch API, WebSocket, Web Worker, Selenium, Jest, PWA (Progressive Web App)

ACTIVIDAD 2. Crea con Google Docs un documento y realiza las explicaciones y capturas de pantalla que demuestren el uso básico del navegador para el desarrollo web. Utilizando las herramientas de desarrollado de tu navegador realiza las siguientes acciones:

2.1 Realiza una aplicación que permite modificar el contenido de un elemento HTML y el CSS de un sitio web. Abre las DevTools para ver los cambios. Realiza un cambio directamente con las DevTools sobre una página web a tu elección con intención de cambiar contenido y estilos. Utiliza las DevTools para mostrar el Modelo de Caja y el diseño en forma de columnas de las propiedades flexbox y grid (busca una página que use grid y/o flexbox o usa una propia)

2.2 Crea una aplicación JS que cree y sea capaz de leer tres cookies (document.cookie) de un usuario. La primera vez que se visita la aplicación el usuario puede seleccionar varias opciones o preferencias (por ejemplo puede ser las distintas opciones de uso de las cookies) y se almacenan en las cookies. La siguiente vez que se visita el sitio (recarga la página) el usuario no necesita seleccionar las opciones ya que se tiene esa información.

2.3 Inspecciona las cookies usadas en tú aplicación usando las DevTools

2.4 Desactiva el uso de la caché, en el apartado red (DevTools)

2.5 Carga la página del www.elpais.com y cuando termine la carga en la pestaña de red de las DevTools elimina toda la información y selecciona la opción XHR. Observa y toma una captura de la información que nos está llegando mediante comunicación asíncrona.

2.6 Utiliza la característica de las DevTools que permite ver una página en distintos dispositivos. (Diseño responsive)

ACTIVIDAD 3. Puesta en marcha.

3.1 Investiga y documenta la puesta en marcha de extensiones de VSCode relacionadas con el desarrollo en Web y con JS (p. ej. JavaScript (ES6) code snippets, git, etc.) detallando sus funcionalidades

3.2 Realiza los ejercicios descritos en el Anexo de este documento

3.3 Ejecuta un script de algún ejercicio anterior desde la consola de visual studio (node + script.js)

3.4 Sincroniza tu proyecto fundamentos-js con un repositorio con tu github (https://github.com/) y publica la página (https://docs.github.com/es/pages/getting-started-with-github-pages/creating-a-github-pages-site)

Se recomienda crear una carpeta para este módulo en tú github e ir creando subcarpetas por cada unidad. Debes aportar un link para ver las actividades

2.- Información de interés

RECURSOS NECESARIOS:

- Ordenador personal con, al menos, 4 Gigabyte de memoria RAM.
- Conexión a Internet.
- Procesador de textos.
- Navegador web.
- Entorno de desarrollo web.
- Software para comprimir los archivos de la Tarea.

WEBS DE INTERÉS:

- Visual Studio Code
- Herramientas del desarrollador (firefox)
- NodeJs
- EsLint
- GitHub
- GitHub Pages

3.- Evaluación. Criterios de evaluación implicados.

RA1: Selecciona las arquitecturas y tecnologías de programación sobre clientes Web, identificando y analizando las capacidades y características de cada una

Los criterios de evaluación implicados del **RA1** son los siguientes:

RA1.a) Se han caracterizado y diferenciado los modelos de ejecución de código en el servidor y en el cliente Web.
RA1.b) Se han identificado las capacidades y mecanismos de ejecución de código de los navegadores Web.
RA1.c) Se han identificado y caracterizado los principales lenguajes relacionados con la programación de clientes Web.
RA1.d) Se han reconocido las particularidades de la programación de guiones y sus ventajas y desventajas sobre la programación tradicional.
RA1.e) Se han verificado los mecanismos de integración de los lenguajes de marcas con los lenguajes de programación de clientes Web.
RA1.f) Se han reconocido y evaluado las herramientas de programación sobre clientes Web.

Rúbrica de la tarea			
Criterios de Evaluación (CE)	**Ejercicio implicado**	**Calificación CE**	**Retroalimentación**
1.a, 1.b, 1.c	Act1		
1.a, 1.b, 1.c, 1.d	Act2		
1.d, 1.e, 1.f	Act3		

ANEXO.

Crea una estructura de carpetas clara y lógica para gestionar múltiples ejercicios de JavaScript, HTML y CSS de manera organizada igual o parecida a la siguiente:

```
fundamentos-js/
├── assets/
│   ├── images/
│   ├── fonts/
│   └── icons/
├── ejercicio1/
│   ├── index.html
│   ├── style.css
│   └── script.js
├── ejercicio2/
│   ├── index.html
│   ├── style.css
│   └── script.js
├── ...
└── ejercicio20/
    ├── index.html
    ├── style.css
    └── script.js
```

Realiza los cambios que consideres necesarios en cada ejercicio para que en index.html aparezca:

- ☐ El enunciado del ejercicio
- ☐ Elementos necesarios para resolver el ejercicio: encabezados, enlaces, formularios, campos
- ☐ Una capa (div) separa que muestre el resultado o salida de cada ejercicio
- ☐ Enlaza correctamente los ficheros .css y .js para que no sean intrusivos (cada parte en un fichero independiente)

CSS:

- ☐ Añade estilos CSS para hacer más vistoso los ejercicios

Documenta cada ejercicio realizando capturas de pantalla dónde se vea el código y la salida del mismo.

Ejercicio 1. Declara variables de varios tipos. Muestra el contenido en la consola junto con el tipo de dato. En este ejercicio utiliza el console.log, console.error, console.info y el console.debug

Ejercicio 2. Declara un array de 100 números. Rellena el array con valores al azar. Muestra el contenido console.table. Realiza un filtrado de datos. Por ejemplo mostrando los valores de un determinado rango

Ejercicio 3. Declara un objeto persona con las propiedades más habituales, usando todos los tipos de datos incluidos un array y otro objeto. Muestra el contenido usando console.log y console.table

Ejercicio 4. Escribe un programa que pida al usuario ingresar dos números y luego muestre la suma. Usa prompt() para pedir datos al usuario y Number() para convertir las entradas a números. Usa console.log() para mostrar el resultado

Ejercicio 5. Crea un programa que verifique si un número ingresado por el usuario es par o impar.

Ejercicio 6. Crea un programa anterior para que se obtenga el número de un campo input.

Ejercicio 7. **Contar vocales en una palabra**
Escribe una función que reciba una palabra y cuente cuántas vocales tiene. Usa un bucle for y la función includes() para verificar si un carácter es una vocal.

Ejercicio 8. Escribe una función que permita generar un número aleatorio entero entre un intervalo. Por ejemplo azar(-5, 10). Salida: 3

Ejercicio 9. **Revertir una cadena**
Escribe una función que tome una cadena como argumento y devuelva la cadena invertida.

Usa el método split() para convertir la cadena en un arreglo, reverse() para invertir el arreglo, y join() para unirlo de nuevo.

Ejercicio 10. Encontrar el número más grande en un array
Crea una función que encuentre el número más grande en un array.

Usa el método Math.max() y el operador ... para "expandir" el array

Ejercicio 11. Escribe un programa que genere 100 números del 1 al 20. Debe mostrar la cuenta de los números que al menos han aparecido una vez.

Ejercicio 12. Filtrar números pares de un array
Crea una función que tome un array de números y devuelva un nuevo array con solo los números pares.Usa filter() y el operador %

Ejercicio 13. Contar palabras en una frase
Escribe un programa que cuente cuántas palabras tiene una frase ingresada por el usuario.

Usa el método split() para dividir la frase en un array de palabras.

Ejercicio 14. Verificar si una palabra es un palíndromo
Escribe una función que verifique si una palabra es un palíndromo (es decir, si se lee igual al derecho y al revés).

Compara la palabra original con su versión invertida usando split(), reverse() y join().

Ejercicio 15. Encontrar los números primos en un rango
Escribe un programa que imprima todos los números primos entre 1 y un número dado por el usuario.

Crea un función llamada esPrimo(num) que reciba num, un número entero

Ejercicio 16. Convertir grados Celsius a Fahrenheit
Escribe una función que convierta grados Celsius a Fahrenheit. La fórmula es:
Fahrenheit = (Celsius * 9/5) + 32

Ejercicio 17. Ordenar un array de números
Escribe un programa que tome un array de números y lo ordene de menor a mayor.

Usa el método sort() con una función de comparación.

Ejercicio 18. Encontrar la palabra más larga en una frase. Escribe una función que encuentre la palabra más larga en una frase ingresada por el usuario.

Usa split() para dividir la frase en palabras, luego recorre el array con un bucle para encontrar la palabra más larga.

Ejercicio 19. Calcular el promedio de un array de números. Escribe una función que reciba un array de números y calcule su promedio.

Usa reduce() para sumar todos los números y luego divide entre la longitud del array.

Ejercicio 20. Convertir una cadena a mayúsculas y minúsculas
Escribe una función que convierta una cadena de texto a mayúsculas y luego a minúsculas, mostrando ambos resultados.

Usa los métodos toUpperCase() y toLowerCase().

Ejercicio 21. Filtrar números mayores que un valor
Escribe una función que tome un array de números y un valor, y devuelva un nuevo array con los números que son mayores que ese valor.

Usa filter() para crear el nuevo array.